

Tutorial: Drawing a Bounding Box with OpenCV

In this tutorial, you will learn how to use the OpenCV library in Python to load an image and draw a bounding box around an object. This is a fundamental step before moving on to object detection with YOLO.

Step 1: Install OpenCV

If you don't have OpenCV installed, you can install it using pip:

```
In [ ]: %pip install --upgrade opencv-python
        %pip install --upgrade matplotlib
```

Verify the installed version number:

```
In [ ]: %pip show opencv-python
```

Step 2: Import Required Libraries

Import the libraries you will use:

```
In [3]: import cv2
        from matplotlib import pyplot as plt
```

Step 3: Load the Image

Use OpenCV's `imread` function to load your image.

Replace the path with the location of your image file.

```
In [7]: # Load the image (change the path to your image file)
        image_bgr = cv2.imread('/Users/eugenio/Documents/Computer_Vision/UC.04 Object De
```

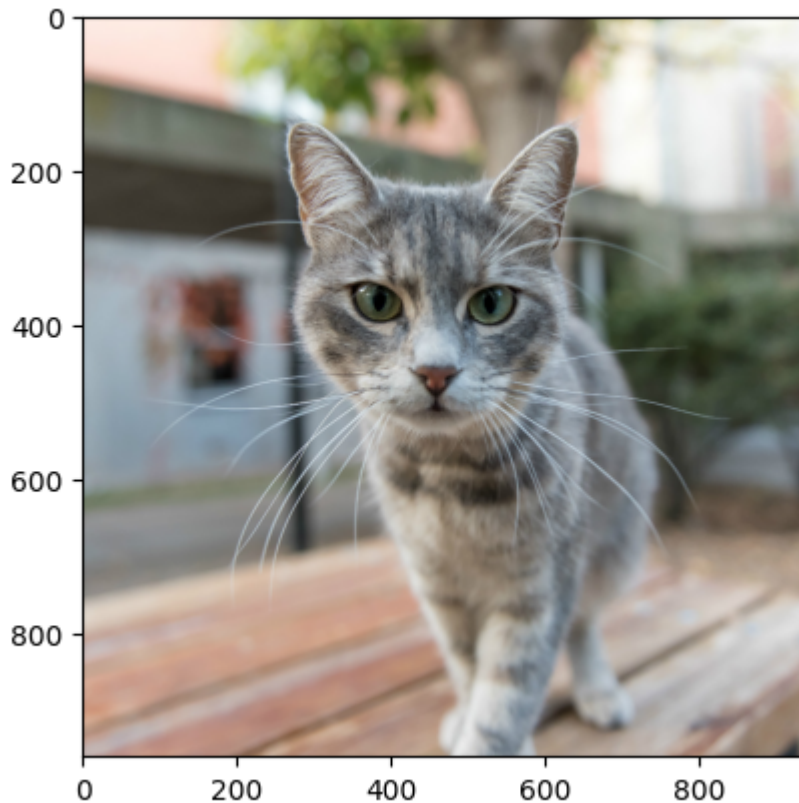
```
In [8]: # Print the shape of the image (height, width, channels)
        print('Image shape:', image_bgr.shape)
```

Image shape: (960, 940, 3)

Step 4: Show the image

```
In [9]: # Convert BGR to RGB for displaying with matplotlib
        image_rgb = cv2.cvtColor(image_bgr, cv2.COLOR_BGR2RGB)

        # Display the image
        plt.imshow(image_rgb)
        plt.show()
```



Step 5: Create the Bounding Box over the Image

To draw a bounding box, use OpenCV's `rectangle` function.

You need to specify the coordinates of the **top-left** point (x_1, y_1) and the **bottom-right** point (x_2, y_2) of the bounding box.

The coordinates should be within the image dimensions

```
In [14]: x1, y1 = 250, 200 # Top-left corner
         x2, y2 = 700, 900 # Bottom-right corner

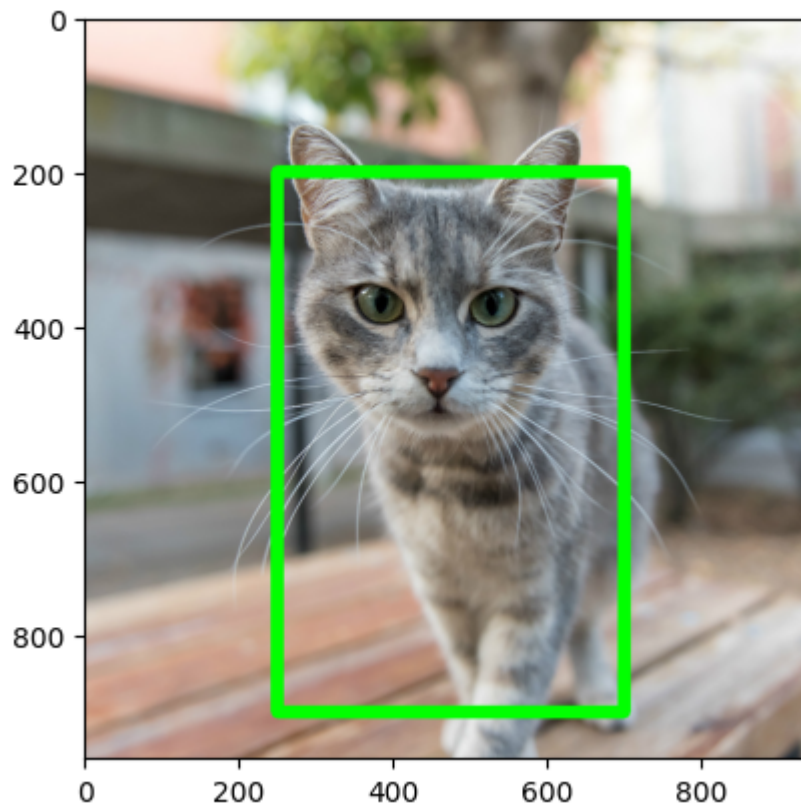
# Draw the bounding box on the input image using cv2.rectangle
# Arguments: image, (x1, y1), (x2, y2), bounding box color in BGR, thickness
image_bgr_box = cv2.rectangle(image_bgr, (x1, y1), (x2, y2), (0, 255, 0), 15) #
```

Step 6: Display the Image and Bounding Box

Finally, use matplotlib to display the image with the bounding box.

```
In [15]: # Convert the image from BGR (OpenCV format) to RGB (matplotlib format)
         image_rgb_box = cv2.cvtColor(image_bgr_box, cv2.COLOR_BGR2RGB)

# Display the image
plt.imshow(image_rgb_box)
plt.show()
```



You should now see your image with a bounding box drawn around the object.

You can adjust the bounding box coordinates as needed to fit the object you want to highlight.

Individual Activity:

1. Load an image containing **at least three different objects from different classes** using `cv2.imread`.

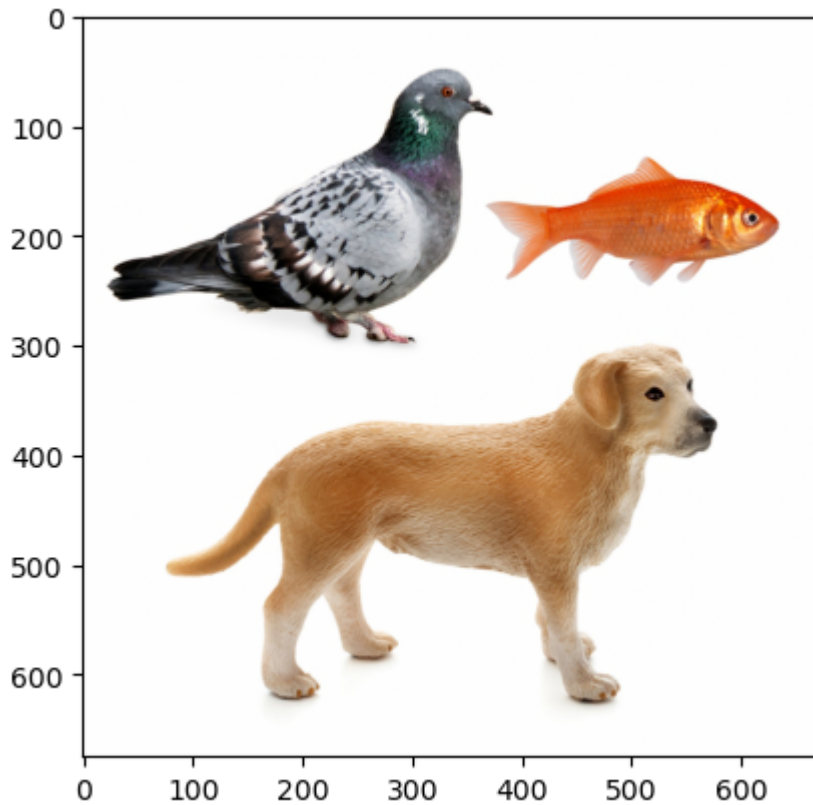
```
In [69]: # Write your code here...

import cv2
from matplotlib import pyplot as plt

image_bgr = cv2.imread('animales.jpg')

# Convert BGR to RGB for displaying with matplotlib
image_rgb = cv2.cvtColor(image_bgr, cv2.COLOR_BGR2RGB)

# Display the image
plt.imshow(image_rgb)
plt.show()
```



2. Draw **different colored bounding boxes** around each object (at least one of the bounding boxes should be yellow). Use a **line thickness** of 5 for both bounding boxes.

In [70]: *# Write your code here...*

```
animal_x1, animal_y1 = 15, 25 # Top-left corner
animal_x2, animal_y2 = 370, 302 # Bottom-right corner

animal_2_x1, animal_2_y1 = 375, 115 # Top-left corner
animal_2_x2, animal_2_y2 = 650, 250 # Bottom-right corner

animal_3_x1, animal_3_y1 = 60, 308 # Top-left corner
animal_3_x2, animal_3_y2 = 585, 635 # Bottom-right corner

# Draw the bounding box on the input image using cv2.rectangle
# Arguments: image, (x1, y1), (x2, y2), bounding box color in BGR, thickness
animal_1_bgr_box = cv2.rectangle(image_bgr, (animal_x1, animal_y1), (animal_x2,
animal_2_bgr_box = cv2.rectangle(image_bgr, (animal_2_x1, animal_2_y1), (animal_
animal_3_bgr_box = cv2.rectangle(image_bgr, (animal_3_x1, animal_3_y1), (animal_
```

3. Add **text labels** above each bounding box (e.g., "Cat", "Ball", etc.) using `cv2.putText`.

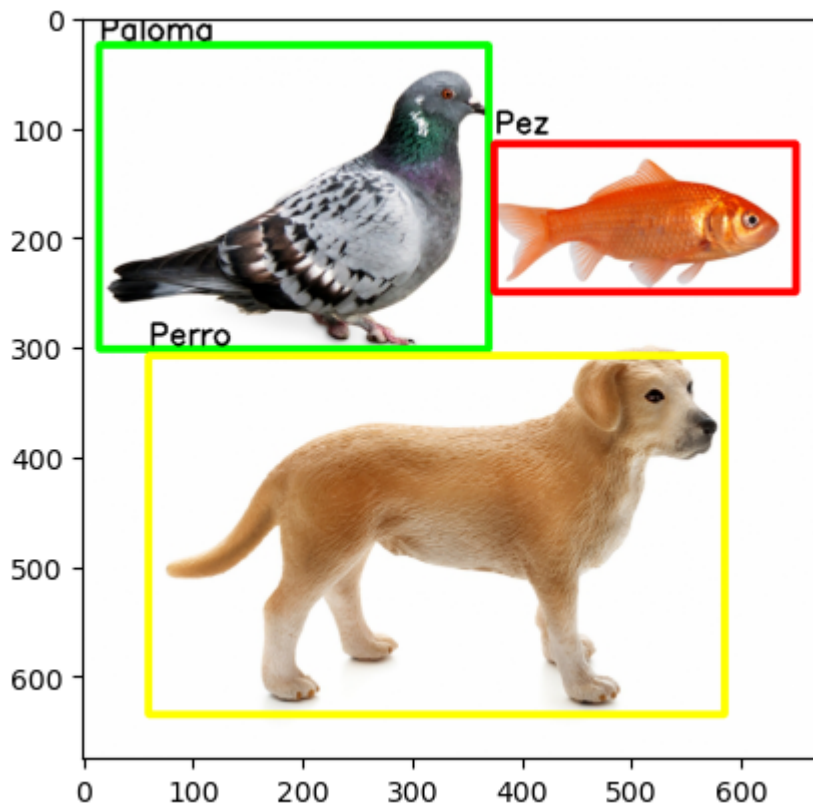
In [71]: *# Write your code here...*

```
cv2.putText(animal_1_bgr_box, 'Paloma', (animal_x1, animal_y1 - 5), cv2.FONT_HERSHEY_SIMPLEX, 1, (0, 0, 0))
cv2.putText(animal_2_bgr_box, 'Pez', (animal_2_x1, animal_2_y1 - 10), cv2.FONT_HERSHEY_SIMPLEX, 1, (0, 0, 0))
cv2.putText(animal_3_bgr_box, 'Perro', (animal_3_x1, animal_3_y1 - 10), cv2.FONT_HERSHEY_SIMPLEX, 1, (0, 0, 0))
```

```
# Convert the image from BGR (OpenCV format) to RGB (matplotlib format)
animal_1_rgb_box = cv2.cvtColor(animal_1_bgr_box, cv2.COLOR_BGR2RGB)
animal_2_rgb_box = cv2.cvtColor(animal_2_bgr_box, cv2.COLOR_BGR2RGB)
animal_3_rgb_box = cv2.cvtColor(animal_3_bgr_box, cv2.COLOR_BGR2RGB)
```

4. Display the **resulting image**.

```
In [72]: # Write your code here...
plt.imshow(animal_1_rgb_box)
plt.imshow(animal_2_rgb_box)
plt.imshow(animal_3_rgb_box)
plt.show()
```



5. Suggest one method (written, no code) to **detect objects automatically** in an image (no manual coordinates). Name the model or technique you would use and briefly explain how it produces bounding boxes.

Se podría utilizar algoritmos de segmentación que permitan identificar las distintas secciones en una imagen y diseñar un algoritmo para dibujar la caja contenedora a su alrededor a partir de los píxeles más extremos de cada una de las regiones detectadas.

Grading Rubric:

Criteria	Points	Description
1. Image Loading	1	Successfully loads an image with at least three objects using <code>cv2.imread</code> . Correct file path and proper handling of image object.

Criteria	Points	Description
2. Bounding Boxes	3	Draws different colored bounding boxes around each object. Boxes have correct size and position, and line thickness is set to 5.
3. Text Labels	2	Adds accurate text labels above each bounding box using <code>cv2.putText</code> . Labels are readable and positioned correctly.
4. Automatic Detection Suggestion	4	Provides a clear and reasonable method to detect objects automatically. Shows understanding of dynamic object detection.

Total: 10 points

Prepared by:

Manuel Eugenio Morocho-Cayamcela